

ATIVIDADE PRÁTICA AVALIATIVA

Sistema IoT de Manutenção Preditiva em Tempo Real

Simulação de Telemetria, Roteamento Dinâmico de Modelos e MLOps

Disciplina / UC:	Engenharia de Dados e MLOps
Caso de Estudo:	Compressores do Brasil S.A.
Aluno:	Nyrx
Data de Entrega:	19 de Agosto de 2026
Formato de Entrega:	Documento em Formato PDF via AVA (Ambiente Virtual de Aprendizagem)

Seção 1: Respostas Teóricas e Analíticas

Questão 1: Arquitetura de Roteamento de Modelos (Teórica)

1.1 Como o código decide qual modelo de Machine Learning acionar?

- A decisão de roteamento é executada na função `prever_ponto_dinamicamente(novo_ponto_de_dados)` através da inspeção do atributo `status_operacao` contido no pacote de telemetria recebido:
- **Validação do Estado:** Compara se o status pertence à lista de operação: `status_motor_rodando = [2, 7, 8, 9, 10, 11, 12, 13, 14]`.
 - **Ramificação Motor Rodando:** Caso verdadeiro, aciona `modelo_rodando.joblib` com dados instantâneos e features derivadas (médias de 7 dias, desvios e desvios-padrão), diagnosticando probabilidade de falhas em 7 dias.
 - **Ramificação Motor Parado:** Caso o status seja 0 ou não pertença à lista, aciona `modelo_parado.joblib` apenas com os dados brutos e horímetros para avaliar a integridade estática durante o repouso.

1.2 Por que o 'chaveamento dinâmico' é uma boa prática industrial em vez de um modelo global?

- **Heterogeneidade de Distribuições:** Variáveis mecânicas (pressão, vibração, vazão, corrente) operam em faixas e dinâmicas radicalmente distintas com o motor sob carga versus em repouso. Um modelo global sofreria com sobreposição e confusão de padrões (*representation interference*).
- **Espaço de Atributos Específico:** Features temporais derivadas só possuem significado físico com fluxo ativo. No estado parado, tornam-se constantes ou ruído irrelevante.
- **Redução da Complexidade de Decisão:** Dividir em subespaços operacionais reduz a não-linearidade do classificador, resultando em fronteiras de decisão mais nítidas e menores taxas de falsos alarmes.
- **Desacoplamento MLOps:** Permite treinar, calibrar e versionar o modelo de operação contínua separadamente do modelo de repouso, facilitando a governança e o deploy sem risco de regressão cruzada.

Questão 2: Análise Crítica de Engenharia de Recursos (Feature Engineering)

2.1 Problemas técnicos e operacionais da aproximação por fatores fixos (0.95 e 0.1)

- No protótipo, os agregados de 7 dias foram aproximados por transformações lineares instantâneas: `_media_7dias = valor * 0.95`, `_std_7dias = valor * 0.1` e `_desvio_7dias = valor - media`. Em ambiente produtivo real, essa simplificação introduz falhas severas:
- **Inexistência de Memória Temporal:** A multiplicação pontual por constante não consulta o histórico prévio. Trata-se de uma escala algébrica que não expressa a tendência real dos 7 dias anteriores.
 - **Multicolinearidade Perfeita:** As features geradas possuem correlação de Pearson de 1.0 com a feature bruta. O modelo Random Forest não recebe nenhuma informação de variabilidade real adicional.
 - **Mascaração de Anomalias:** Em pico súbito de pressão (ex.: subida repentina para 25 bar), a média 'aproximada' sobe instantaneamente para 23.75 bar, quando a média real ponderada de 7 dias deveria permanecer estável. Isso impede a detecção de variações anômalas.
 - **Desvio de Treinamento-Serviço (Training-Serving Skew):** Se o modelo foi treinado com séries históricas verdadeiras e recebe aproximações sintéticas em produção, a acurácia degrada severamente.

2.2 Arquitetura em Nuvem Ideal para Agregados Temporais de 7 Dias

Camada	Tecnologias	Responsabilidade no Pipeline
1. Ingestão	Apache Kafka / AWS Kinesis	Bufferiza telemetria com alta taxa de transferência, tolerância a falhas e ordenação por timestamp.
2. Stream Proc.	Apache Flink / Spark Streaming	Aplica janelas deslizantes (<i>Sliding Windows</i> de 7 dias com slide de 10 min) calculando média, desvio e variância reais.
3. Feature Store	Feast / SageMaker Feature Store	Armazena features online (baixa latência via Redis/DynamoDB) e offline (treinamento via Parquet/S3).
4. Time-Series	TimescaleDB / InfluxDB	Persistência histórica de longo prazo para re-análise, consultas analíticas e auditoria de sensores.
5. Model Serving	Triton / FastAPI + Docker	Recebe o vetor de features pré-computado e responde a predição com latência inferior a 50ms.

Questão 3: Simulação de Latência e Gargalos no Streamlit

3.1 Qual fenômeno do mundo real da comunicação IoT é simulado?

A expressão `tempo_de_espera = intervalo_base_segundos + random.uniform(0, atraso_iot_max_segundos)` simula o **Jitter de Rede** (variação estatística da latência de transmissão) e as incertezas de propagação de enlaces industriais sem fio (redes Celulares 3G/4G/5G e LoRaWAN):

- **Restrições de Duty Cycle regulatório:** Limitações legais de tempo de transmissão por hora em faixas ISM (LoRaWAN).
- **Contenção e Colisão de Pacotes:** Múltiplos sensores concorrendo pelo mesmo gateway geram retransmissões automáticas (*Exponential Backoff*).
- **Flutuações de Sinal e Interferência Eletromagnética:** Atenuação em ambientes de fábrica com motores de alta potência e estruturas metálicas.

3.2 Estratégias no Streamlit para atualização sem bloquear a thread principal

O laço bloqueante `while True` com `time.sleep()` congela a thread de execução do Streamlit, impedindo a interação fluida com botões e menus. Para solucionar este problema, recomenda-se:

- **Uso de `st.fragment` (Streamlit ≥ 1.33):** Isola a função de atualização com o decorador `@st.fragment(run_every=timedelta(seconds=10))`. Apenas o container do dashboard é re-renderizado periodicamente em background, mantendo toda a interface responsiva.
- **Componente `streamlit-autorefresh`:** Delega o temporizador para o frontend do navegador via JavaScript, disparando re-runs assíncronos sem travar a thread Python.
- **Arquitetura com WebSockets / SSE:** Em sistemas industriais avançados, o frontend consome um broker de mensagens assíncrono via WebSocket, eliminando polling ativo.

Seção 2: Código Refatorado e Comprovação de Execução (Questão 4)

Implementação da Tabela de Histórico Recente no Streamlit

Conforme solicitado na Questão 4, o arquivo `Teste_Compressor.py` foi modificado para armazenar em sessão (`st.session_state.historico_leituras`) as últimas 5 leituras recebidas e exibi-las em uma tabela estruturada logo abaixo dos cartões de status do compressor.

```
# 1. Importação adicional
from datetime import datetime

# 2. Inicialização do Histórico e Placeholder na Interface
st.markdown("---")
st.markdown("### ■ Tabela de Histórico Recente (Últimas 5 Leituras)")
placeholder_historico = st.empty()

if 'historico_leituras' not in st.session_state:
    st.session_state.historico_leituras = []

# 3. Bloco executado a cada nova leitura recebida no loop de simulação
horario_leitura = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
status_texto = "Rodando" if current_status in status_motor_rodando else "Parado"
resultado_predicao = "Possível Falha" if "Falha" in label else "Normal"

nova_leitura = {
    'Horário': horario_leitura,
    'Status Operação': status_texto,
    'Pressão Saída': round(novo_ponto['pressao_saida'], 2),
    'Predição': resultado_predicao
}

# Insere no início para exibir os dados mais recentes no topo
st.session_state.historico_leituras.insert(0, nova_leitura)

# Mantém estritamente as últimas 5 leituras
if len(st.session_state.historico_leituras) > 5:
    st.session_state.historico_leituras = st.session_state.historico_leituras[:5]

# Renderização tabular dinâmica
with placeholder_historico.container():
    df_historico = pd.DataFrame(st.session_state.historico_leituras)
    st.dataframe(df_historico, use_container_width=True, hide_index=True)
```

Comprovação Gráfica de Execução no Navegador (Questão 4)

A imagem a seguir comprova a execução da aplicação Streamlit no navegador, demonstrando o recebimento contínuo de telemetria dos sensores industriais, a classificação em tempo real do modelo de Machine Learning e a exibição tabular dinâmica das últimas 5 leituras com colunas de Horário, Status de Operação, Pressão de Saída e Resultado da Predição:



Figura 1: Interface do Streamlit em execução no navegador exibindo os cartões de diagnóstico e a nova Tabela de Histórico Recente (Questão 4).

Conclusão e Validação MLOps:
O protótipo foi validado com sucesso. Todos os requisitos de ambiente (requirements.txt, venv), geração de modelos (treinarmodelo.py), chaveamento dinâmico de inferência e aprimoramento da interface com a Tabela de Histórico Recente de 5 posições foram plenamente atendidos e homologados para entrega.